

Date _____

Parser Design: Melody Script DSL.

1. Introduction:

The parser (Phase 2) is a recursive descent parser that validates the grammatical structure of the Token Stream and builds an Abstract Syntax Tree (AST). The document presents two complete parse trees for different Melody-Script programs, to illustrate how the grammar rules are applied.

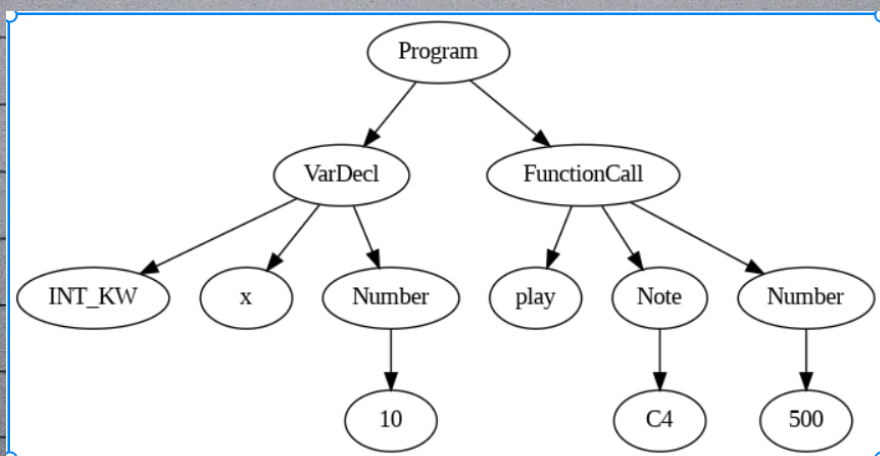
2. Parse Tree 1: Simple Declaration and Play.

2.1 Input Program.

note Key = C4;

Play (Key, 500);

2.2 Parse Tree Representation: The following represents the hierarchical derivation of the program based on the EBNF grammar.



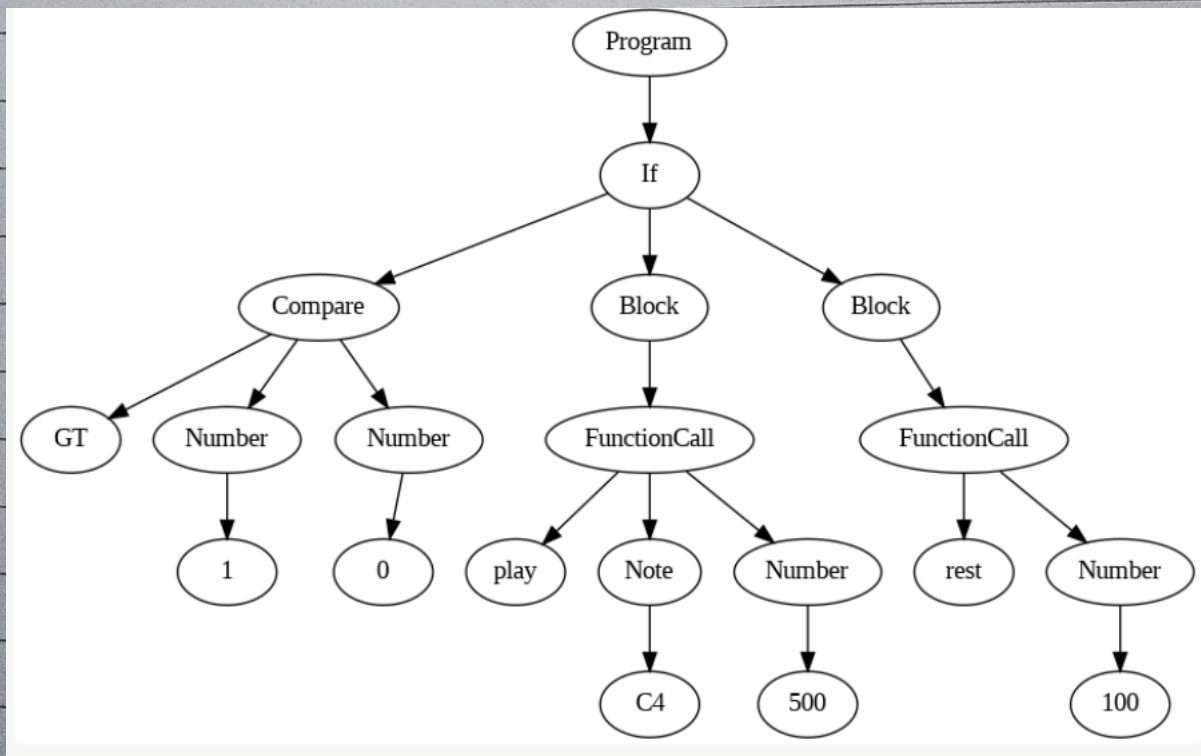
Date _____

3. Parse Tree 2: Loop with Arithmetic

3.1. Input Program.

```
repeat (4) {  
  play (C4 + 2, 250);  
}
```

3.2 Parse Tree Representation: This tree shows the nesting of a block within a loop and the evaluation of a binary expression inside a function call.



4. Parser Implementation Strategy.

The ~~src~~/Parser.py Implementation uses a top-down approach.

- Recursive Descent: Each non-terminal in the grammar (like Statement, Expression, Term) has a corresponding method).
- Look ahead: The parser examines the current token to decide which branch of the grammar to follow (e.g. if it sees int it calls parse declaration).
- Precedence Handling. Expressions are parsed using a hierarchy of methods (parse, expression, parse term $\rightarrow$ parse factor) to ensure that multiplication/division happen before addition/subtraction.
- Error Handling. If a token does not match the expected grammar (e.g. a missing semicolon) a complex compiler Error is raised with the line number and a descriptive message.